QUALITY ASSURANCE TESTING DOCUMENTATION (QATD)

FOR TEAM BAKKUTTEH

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

Table of Content

Document Control **3**
PRELIMINARY ROUND (Test Strategy & Planning) **3**
1. Scope & Requirements Traceability 3
2. Risk Assessment & Mitigation Strategy 3
3. Test Environment & Execution Strategy 5
4. CI/CD Release Thresholds & Automation Gates 5
5. Test Case Specifications (Drafts) 6
6. Test Strategy & Plan Sign-Off 7

© UMHackathon 2026

Email: umhackathon@um.edu.my

Document Control

Field	Detail
System Under Test (SUT)	*UMHackathon 2026 - Team **Bakkutteh*** *Crypto-Integrated Financial Intelligence Advisor (CIFA)*
Team Repo URL	*https://github.com/jiexin2006/bakkutteh.git*
Live Deployment URL (optional for preliminary round)	*None, as we used localhost*

Objective:

The primary objective is to ensure that CIFA provides accurate mathematical surplus allocations and reliable AI-driven "Decision Intelligence" for users.

PRELIMINARY ROUND (Test Strategy & Planning)

1. Scope & Requirements Traceability

*This section aligns the testing connected back to specific user requirements via **Requirement Traceability Matrix".** So, it ensures every test has been conducted to prevent bugs/failure in products for that specific requirements and unplanned feature creep.*

1.1 In-Scope Core Features

- ***Real-Time Cryptocurrency:*** *Integrates live Bitcoin rates to suggest "Buy/Hold/Sell" moments*
- ***Fixed Deposit Comparison:*** *Ranks top Malaysian banks by rate and takes into account their minimum threshold (normally ranging between RM 500 and RM 1000)*
- ***EPF Health Scoring:*** *The user's current EPF balance is compared with the current EPF standard of their category (basic, adequate, and enhanced) and is decided on whether the user is currently "way behind", "behind" or "on track".*
- ***Conversational Reasoning:*** *Z.AI explains how and why money should be allocated to the EPF, FD, and Crypto based on current market data and the unique user financial data*

1.2 Out-of-Scope

- **Stock Market Integration:** Real-time stock market trading
- **Identity Management:** User authentication

© UMHackathon 2026

Email: umhackathon@um.edu.my

2. Risk Assessment & Mitigation Strategy

Technical Risk	Likelihood (1–5)	Severity (1–5)	Risk Score (L×S)	Mitigation Strategy	Testing Approach
Z.AI API Timeout	3	4	12 (High)	Implemented a 2-tier fallback strategy: First fallback to Gemini 1.5 Flash, second fallback to a static rule-based advisory if both fail.	Simulated a 504 Timeout and verified the backend automatically switched to Gemini within 1 second.
Dependency Conflict (Protobuf)	4	4	16 (High)	Used defensive imports (importing libraries inside functions only when needed) and relaxed requirements.txt constraints to allow for cross-version compatibility between TensorFlow and Google Cloud libs.	Verified the FastAPI server successfully starts and handles requests even if some secondary ML/Cloud libraries are temporarily missing.
API Key Exposure (Security)	2	5	10 (Medium)	Migrated sensitive keys from .env to Google Cloud Secret Manager. Keys are fetched at runtime via a secure helper and never logged to the console.	Successfully fetched ILMU_API_KEY from GCP while the local .env entry was commented out.
Hallucinated Market Data	3	4	12 (High)	Used Contextual Grounding: Injected a verified JSON of Malaysian bank rates into the prompt and used a strict system instruction forbidding the AI from using any banks not in the list.	Performed a "Hallucination Test" (AI-02) to verify the AI only recommended banks provided in the dynamic context (e.g., Maybank, CIMB).

© UMHackathon 2026

Email: umhackathon@um.edu.my

Risk Assessment Scoring Criteria:

Likelihood (1-5)		*Severity (1-5)*	
1	*Rare*	*1*	*Impact is Negligible*
2	*Unlikely*	*2*	*Impact is Minor*
3	*Possible*	*3*	*Moderate Impact*
4	*Likely*	*4*	*Major Impact*
5	*Almost Certain*	*5*	*Critical Failure of the system*

Risk Score = Likelihood × Severity

Risk Level Reference:

Risk Score	***Risk Level***	***Recommended Action***
1 – 5	*Low*	*Monitor only. Acceptable risks.*
6 – 10	*Medium*	*Mitigate + Testing*
11 – 15	*High*	*Must need mitigating and through testing is required*
16 – 25	*Critical*	*Priority is Highest. Need extensive level of testing.*

3. Test Environment & Execution Strategy

Unit Test

- Scope: Financial Input Validation (AdvisoryRequest schema), Bitcoin Analyzer logic, and EPF Calculator mathematics.
- Execution: Tests are written using Pytest. Execution occurs locally during development and is triggered via GitHub Actions on every push.
- Isolation: We use unittest.mock to isolate business logic. External services like Z.AI, Gemini, and yfinance are mocked to ensure tests are fast and don't consume API tokens.
- Pass Condition: All validation rules (e.g., "Age must be 18–120") must trigger the correct 422 Unprocessable Entity response for negative cases.

Integration Test

- Scope: The complete "Advisory Orchestration" flow—connecting the Frontend UI to the FastAPI Backend and the AI Engines.
- Execution: Performed after successful unit testing of both the Frontend and Backend components.

© UMHackathon 2026

Email: umhackathon@um.edu.my

- Workflow: We test the real interaction between the Backend and GCP Secret Manager to ensure keys are fetched correctly, followed by a live call to the AI engine to verify the parsing of the JSON response.
- Pass Condition: A valid, correctly formatted strategy is successfully rendered on the Dashboard for a given user profile.

Test Environment (CI/CD)

- Local Testing: Manual testing on localhost:5173 (Vite) and localhost:8000 (uvicorn) to verify UI/UX and hot-reloads.
- Staging/CI: GitHub Actions automatically runs the Pytest suite on every Pull Request to the main branch.
- Automated Pipeline: The CI pipeline enforces that the code must be linted and all unit tests must pass before the "Merge" button is enabled.

Regression Testing & Pass/Fail Rules

- Execution Phase: A full regression sweep across all 4 core test cases (Happy Path, Negative Validation, Performance, and Fallback) is performed before every deployment.
- Pass/Fail Condition: A test is only "Pass" if the AI's JSON output matches our expected schema 100%. If the AI adds conversational text that breaks our parser, it is marked as a "Fail" and the parse_model_json utility is updated.
- Continuation Rule: Validation tests (TC-02) are considered "Blockers." If input validation fails, all subsequent testing is halted until the gatekeeper logic is fixed.

Test Data Strategy

- Manual Data: We maintain 3 "Golden Profiles":
    1. "The Behind Saver": (Age 25, low EPF) to test safety-first logic.
    2. "The Aggressive Investor": (Age 35, high surplus) to test crypto-heavy allocations.
    3. "The Invalid Case": (Age 200, Salary -500) to test rejection logic.
- Automated Data: We use JSON seed files in the data/ directory to provide the system with "Verified Market Rates" during testing.

Passing Rate Threshold

- Overall Threshold: A minimum of 90% of all test cases must pass.
- Critical Threshold: 100% pass rate is mandatory for Security (Secret Manager connectivity) and Safety Validation (Pydantic constraints). No advisory can be generated if the safety guardrails are compromised.

© UMHackathon 2026

Email: umhackathon@um.edu.my

4. CI/CD Release Thresholds & Automation Gates

This section is mainly for defining metrics & threshold for success criteria. So, if a certain threshold is not met, the pipeline blocks the forward transition.

4.1 Integration Thresholds (Merging to Main)

The primary needs of this threshold to be crucial is because the main branch must be a stable source of truth. So, it needs to be achieved through continuous integration checks.

Checks	***Requirements***	***Project***	***Pass/Failed***
Automatic Build	***Zero Build Error***	***5 Errors***	***Failed***
Unit Tests	***100% Passing Rate***	***100%***	***Passed***
Code Quality	***Zero Linting Error***	***97%***	***Failed***
Test Coverage	***Minimum 81.2%***	***79%***	***Failed***

4.2 Deployment Thresholds (Pushing to Production)

Checks	***Requirements***	***Project***	***Pass/Failed***
Regression Test	***Minimum 90%***	***91%***	***Passed***
AI Output Pass Rate	***Minimum of 80% of documented prompt***	***78%***	***Failed***
Critical Bugs	***Zero P0/01 Bugs***	***2***	***Failed***
API Performance	***Response Time < 800ms***	***450ms***	***Passed***
Security	***API keys are not exposed***	***Clean***	***Passed***

© UMHackathon 2026

Email: umhackathon@um.edu.my

5. Test Case Specifications

Test Case ID	Test Type & Mapped Feature	Test Description	Test Steps	Expected Result	Actual Result	Status
TC-01	Happy Case (Entire Flow): Financial Advisory	Verify that a new user can complete onboarding and receive a personalized AI investment strategy.	1. Open App and complete Onboarding (Age 30, Salary 5k). 2. Click "Generate Advisory". 3. Backend fetches live BTC data. 4. AI reasoning engine generates plan. 5. Dashboard renders the strategy.	Strategy is received within 15s. Allocation percentages sum to 100%. Specific Malaysian banks are recommended for FDs.	Strategy received in 12.4s. Allocation sum is 100%. UI rendered all action items correctly.	Passed
TC-02	Specific Case (Negative): Unreasonable Financials	Verify the system blocks inputs where expenses are mathematically impossible (e.g., 5x salary).	1. Navigate to Profile Settings. 2. Enter Salary: RM 3,000. 3. Enter Expenses: RM 15,000. 4. Attempt to click "Save".	Save button is disabled. Red error message appears: "Monthly expenses are unreasonably high." API call is blocked.	Button disabled. Correct error message displayed in red. Backend also returned 422 if manual POST was attempted.	Passed

© UMHackathon 2026

Email: umhackathon@um.edu.my

TC-03	NFR (Performance): Concurrent AI Load	Ensure the FastAPI backend remains responsive when multiple users request advice simultaneously.	1. Use load-testing tool (e.g., k6). 2. Send 10 concurrent POST requests to /api/advisory. 3. Record latency and error rate.	Average response time < 20s (including AI latency). 0% Error rate.	Average response time: 16.2s. 0% Error rate. System handled concurrent async tasks well.	Passed
TC-04	NFR (Architecture): AI Fallback Resilience	Verify that the system automatically switches to the Gemini fallback if the primary ZAI engine is offline.	1. Revoke/Change ILMU_API_KEY to simulate failure. 2. Trigger a new advisory request. 3. Monitor backend logs for fallback trigger.	Backend logs "PRIMARY MODEL FAILED". System calls Gemini fallback. Valid advisory is returned to user.	Logs confirmed "Switching to Gemini Fallback". User received valid strategy with no visible error.	Passed

6. AI Output & Boundary Testing (Drafts)

6.1. **Prompt/Response Test Pairs**

Document at least 3 Prompts/response test pairs. For each of them, do define the acceptance criteria - what a correct, acceptable or a failing response may look like for your use case.

Test ID	Prompt Input (Simplified Context)	Expected Output (Acceptance Criteria)	Actual Output	Status
AI-01	JSON Structural Integrity: User profile + Market	The output must be strictly valid JSON. No	The model returned a clean JSON object. The	Pass

© UMHackathon 2026

Email: umhackathon@um.edu.my

	rates + Instruction: "Return RAW JSON only."	markdown code blocks (```json), no intro ("Here is your..."), and no hallucinated fields. Must contain overall_strategy and action_plan.	system's parse_model_json utility successfully extracted the fields without any regex cleanup required.	
AI-02	Guardrail Check (RM 500 Rule): User has a monthly surplus of RM 350.	The AI must NOT recommend a Fixed Deposit (FD), as the bank minimum is RM 500. It must justify why FD is 0% and suggest "Cash on Hand" instead.	Action Plan: category: "FD", percentage: "0%", action: "Maintain in Savings", reasoning: "Monthly surplus is below the RM 500 minimum for high-yield FD placement."	Pass

6.2. Oversized/Larger Input Test

This test ensures that the system handles "unreasonable" financial inputs gracefully without overwhelming the AI engine.

Fields	Details
Maximum Input Size	Enforced at the API level via Pydantic: Salary max RM 1M, Assets max RM 20M, Age max 120.
Input used while testing	Inputting a Salary of RM 10,000,000 (10x the limit) and an Age of 150.
Expected Behavior	Reject: The system should block the request and prompt the user to enter realistic data.

© UMHackathon 2026

Email: umhackathon@um.edu.my

Actual Behavior	The Frontend displayed red inline error messages and disabled the "Generate" button. The Backend returned a `422 Unprocessable Entity` error.
Test Output	`"msg": "ensure this value is less than or equal to 1000000"`
Status	Pass

6.3. Adversarial/Edge Prompt Test

Scenario: Testing a "Negative Surplus" scenario where a user inputs expenses significantly higher than their income to try and force a high-risk recommendation.

- Prompt/Input: Salary: RM 3,000 | Expenses: RM 15,000.
- GLM Response: If this reached the GLM, it would be forced into "Survival Mode" by the System Instructions, recommending 100% Cash-on-Hand.
- System Handling: Our backend model_validator in api.py detects this discrepancy (Expenses > 3x Salary) and rejects the request with a custom error message: "Monthly expenses are unreasonably high relative to salary," preventing the AI from even being triggered.

6.4. Hallucinating Handling

We mitigate hallucinations through three primary layers of "Strict Grounding":

1. Dynamic Contextual Grounding: We do not ask the AI "What are the best FD rates?" Instead, we fetch the rates from our verified JSON data and inject them into the prompt. The AI is strictly instructed: "Do NOT mention any bank or rate NOT found in the 'verified_market_rates' list."
2. JSON Schema Enforcement: We use a parse_model_json utility that uses regex-like locators to find valid JSON blocks. If the AI hallucinates conversational text or "extra thoughts" outside the JSON, our system automatically strips them out.
3. Cross-Model Validation: If the primary GLM (Z.AI) returns a response that fails our structural validation, the system automatically switches to the Gemini Fallback. This secondary check ensures that a hallucinating response from one model doesn't reach the user.

© UMHackathon 2026

Email: umhackathon@um.edu.my